

1. Data Preprocessing

Our raw dataset contains four components: tick-level order book records, tick-level transaction records, tick-level Chainlink oracle updates, and a contract-level metadata file (`trader_week`). Because our study is not aimed at modeling ultra-high-frequency execution itself, but rather at understanding the information content of prediction market activity, we aggregate the raw tick data to a one-second frequency. The final dataset is a contract-by-second panel, where each observation corresponds to one contract at one second during the life of a 15-minute market.

For the order book, we retain the top five bid and ask levels and keep the last observed snapshot within each one-second interval. We then compute the total displayed depth across the top five levels on both sides. An important feature of the prediction market design is that the “up” and “down” contracts are structurally mirrored, since they correspond to complementary payoff states of the same binary event. As a result, quote-level information is largely redundant across the two sides. We therefore retain only the “up” market in the final order book dataset.

Transaction data are handled differently. Since the trade records reflect taker-initiated executions, trading activity in the “up” and “down” markets is not mechanically symmetric even when the two contracts are economically linked. To preserve potentially informative order-flow differences, we keep transaction data for both sides and aggregate all trades within each second into trade size, trade count, notional value, and side-specific VWAP measures.

The Chainlink data provide the reference price path of the underlying asset, Bitcoin. We resample the oracle series to one second using the last available update and forward-fill missing seconds. Finally, we merge these market features with contract-level metadata from `trader_week`, including strike prices and realized outcome labels. This construction allows us to jointly examine order book conditions, transaction flow, and underlying Bitcoin price dynamics within each contract horizon.

2. Feature Engineering

```
['timestamp', 'slug', 'Net_OFI', 'macro_return_15min', 'macro_rv_15min',  
'macro_return_1h', 'macro_rv_1h', 'time_to_expiry', 'moneyness', 'ewma_rv_1s',  
'Up_price_momentum_15s', 'sigma_annualized', 'bsm_theoretical_price', 'bsm_delta',  
'Up_OBI', 'Up_spread', 'Up_Mid_Price', 'Down_Mid_Price', 'Up_micro_price',  
'Net_Nominal_Flow', 'Net_Flow_ema_15s', 'Net_Flow_ema_60s', 'CVD_Net_Flow',  
'Whale_Imbalance_ma_10s', 'Price_Discovery_Spread', 'Up_BUY_Urgency',  
'Down_BUY_Urgency', 'BSM_Mispricing', 'Micro_Price_Imbalance']
```

We organize our candidate features into three broad groups. These groups are designed to capture three complementary dimensions of information in the market: contract-level valuation, short-run trading pressure, and the broader Bitcoin regime. Taken together, they reflect our view that the pricing dynamics of a 15-minute prediction market contract are shaped not only by its payoff structure, but also by order-flow conditions and the state of the underlying market environment.

First, because the 15-minute prediction market contract can be viewed as a short-horizon binary contingent claim, option-pricing-related variables play a central role in our design. We therefore

construct time-to-expiry, moneyness, short-horizon realized volatility, theoretical price, delta, and pricing-dislocation measures based on the underlying Bitcoin path. These variables are intended to capture the contract's evolving payoff structure and sensitivity to changes in the underlying asset, and thus provide a valuation-based benchmark for interpreting market prices.

Second, we build a set of market microstructure features from both the order book and the transaction data. The order book captures standing liquidity and cumulative quote pressure, while transactions capture more instantaneous aggressive trading activity. Based on these two sources, we construct features related to quote imbalance, spread, micro-price deviations, momentum, order-flow imbalance, nominal-flow pressure, trade urgency, and proxies for unusually large trades, or “whale” activity. These features are designed to capture short-run information arrival and directional pressure that may not be fully reflected in pricing-based variables alone.

Third, we augment the contract-level feature set with macro-regime variables derived from the recent Bitcoin price path, including rolling returns and realized volatility over longer horizons. In the current implementation, these regime variables are constructed over 15-minute and 1-hour windows to summarize the broader market environment in which each contract is traded. This layer is intended to capture whether a contract is being traded in a relatively calm, trending, or high-volatility regime.

Overall, our feature engineering strategy is intentionally expansive at the outset. Rather than imposing strong priors on which signals should matter *ex ante*, we begin with a broad candidate set spanning valuation, microstructure, and regime information. In a second step, we apply feature filtering to remove mechanically redundant and weakly informative variables, producing a more parsimonious feature set for downstream analysis.

3. Sample Refinement and Feature Selection

In practice, sample construction, feature engineering, and data cleaning are closely intertwined. Some filters can be applied directly to the raw contract data, while others are naturally defined only after derived variables such as volatility, spread, or rolling flow signals have been constructed. We therefore treat the overall preprocessing pipeline as a staged but iterative procedure.

We first remove structurally incomplete contracts. In particular, because a large part of our pricing-related feature set depends on the underlying Bitcoin reference path, we exclude contracts for which Chainlink updates are missing or stale to the extent that the contract-level data cannot support reliable construction of volatility, moneyness, or theoretical-pricing variables. This step is applied prior to the main feature-engineering stage.

We then further refine the sample to focus on economically meaningful and sufficiently active trading periods. First, we remove warm-up intervals at the beginning of the sample or contract life, since rolling and exponentially weighted features are not yet fully formed in those periods. Second, we trim intervals near the end of the 15-minute market when trading activity may become extremely sparse and quoted prices may cease to be informative. As a result, the final prediction sample is restricted to active market states in which short-horizon price movements

remain economically meaningful and the constructed features are based on sufficiently mature information.

After constructing the full candidate feature set, we apply a two-stage feature-filtering procedure. The first stage is deterministic: we remove raw variables whose information is already embedded in more structured derived factors. This step is intended to reduce mechanical redundancy and limit multicollinearity among closely related inputs. The guiding principle is that once higher-level pricing, flow, or liquidity factors have been explicitly constructed, retaining all underlying raw components is unlikely to add proportional predictive value.

The second stage is model-based. We use LightGBM as a computationally efficient supervised screening device and rank candidate variables according to gain-based feature importance. To improve robustness, this selection procedure is combined with grid search and repeated across multiple random seeds, so that the final retained variables are not driven by a single partition or initialization. We then further prune the selected set by examining cross-feature dependence and removing highly redundant variables using a graph-based procedure. The objective is to retain features that are jointly informative, relatively stable across resampling schemes, and not excessively collinear with one another.

This design reflects our broader modeling philosophy. We begin with an intentionally expansive feature set in order to avoid imposing strong ex ante priors on which signals matter in this market. We then impose discipline through feature filtering, retaining only those variables that combine predictive relevance, robustness, and limited redundancy. Feature selection is conducted within the training sample to avoid leakage of information from held-out contracts into the final specification.

We also considered unsupervised dimensionality-reduction methods such as PCA and autoencoders. However, because our candidate variables are already economically structured and the prediction target is a very short-horizon price movement, we prioritize supervised filtering methods that preserve interpretability and predictive relevance. In particular, an autoencoder is trained to reconstruct input structure rather than maximize forecasting power, and in a high-frequency setting this objective may allocate substantial capacity to high-variance microstructure fluctuations that do not generalize well out of sample.

4. Model Selection

1. Data split: train, validation, and test

Train range: 2025-12-01 06:01:31+00:00 -> 2025-12-12 11:37:08+00:00
Val range: 2025-12-12 11:37:09+00:00 -> 2025-12-13 17:02:57+00:00
Test range: 2025-12-13 17:02:58+00:00 -> 2025-12-15 03:14:24+00:00

Our prediction task is constructed on a pooled contract-by-second panel, where each observation corresponds to one contract at one second and the target is the 5-second-ahead return. To preserve the temporal structure of the data and avoid look-ahead bias, we split the sample in

chronological order rather than randomly. Specifically, the most recent portion of the sample is held out as the test set, while the earlier portion is used for model development. Within the development sample, we further separate training and validation data in a forward-looking manner. This design ensures that model fitting, hyperparameter tuning, and performance evaluation all respect the time ordering of the market, so that information from later periods is never used to inform earlier predictions.

For sequence models, the split is implemented before sequence construction is finalized, so that each 60-second input window and its corresponding 5-second-ahead label remain entirely within the same sample partition. In addition, sequence generation is performed separately within each contract (`slug`) to prevent the tail of one contract from being mechanically linked to the beginning of another. As a result, the final train, validation, and test sets reflect a realistic forecasting environment in which the model is always evaluated on genuinely later market observations.

Although the overall sample is partitioned in chronological order, the training sequences are shuffled across batches during optimization. This does not compromise the temporal structure of the task, since time ordering is preserved within each 60-second input window; the shuffling only affects the order in which training examples are presented to the optimizer.

2. Model Architecture and Selection: An Evolutionary Ablation Framework

In high-frequency market prediction, model choice determines not only predictive accuracy, but also the type of market structure the model is capable of exploiting. Rather than selecting a complex architecture a priori, we adopt an evolutionary, ablation-style framework in which models are introduced sequentially to isolate the contribution of different modeling capabilities. In particular, we distinguish between three dimensions of improvement: nonlinear feature interaction, continuous representation learning, and explicit temporal memory. This progression allows us to assess whether performance gains arise from richer function approximation alone or from the ability to model short-horizon path dependence.

2.1 Cross-Sectional Baseline: XGBoost

We begin with XGBoost as a strong cross-sectional benchmark. Tree-based ensemble models are widely used in quantitative finance for tabular prediction problems because they are effective at capturing nonlinear interactions among engineered variables and are generally robust in noisy structured-data settings. In our context, XGBoost provides a natural benchmark for evaluating how much predictive content can be extracted from the engineered feature set without explicitly modeling sequential dependence.

At the same time, XGBoost is fundamentally a cross-sectional model. It evaluates the market state at time `t` as a static feature vector. Although some of the engineered inputs, such as rolling momentum or EWMA volatility, summarize recent history, the model itself does not maintain an internal memory of how the market arrived at the current state. It therefore represents our strongest benchmark without explicit temporal awareness.

2.2 Neural Bridge: Multi-Layer Perceptron (MLP)

To separate the effect of neural nonlinear representation learning from the effect of temporal memory, we introduce a multi-layer perceptron (MLP) as an intermediate control architecture. This comparison is important for identification: if a recurrent model outperforms XGBoost, the improvement could arise either from the neural architecture's ability to learn smoother and more flexible nonlinear mappings, or from its ability to model sequential structure. The MLP helps disentangle these two effects.

Like XGBoost, the MLP operates on a single-time-step cross section. It maps the engineered feature vector observed at time t directly to the future return target, without any explicit recurrent state. Thus, it remains a cross-sectional model, even though some input variables contain hand-engineered temporal summaries. Its role is not to capture time evolution directly, but to test whether dense neural feature interaction alone improves upon the tree-based baseline. In this sense, the MLP serves as the neural baseline against which the additional value of sequence modeling can be assessed.

2.3 Introducing Temporal Dependence: Standard RNN

After establishing the cross-sectional baselines, we extend the input into a 60-second rolling sequence and introduce a standard recurrent neural network (RNN) as the first temporal model. The purpose of this step is to allow the model to process the recent market path rather than only the current cross section. In principle, this architecture should be able to detect whether the same current market state has different implications depending on the path by which it was reached.

Empirically, however, the standard RNN performs poorly in our setting. A likely explanation is that the predictive structure in high-frequency crypto microstructure is both noisy and locally path-dependent. Over a 60-step input window, a vanilla RNN must repeatedly compress past information into a single evolving hidden state. In practice, earlier informative patterns can be attenuated or overwritten by later, less informative fluctuations. This hidden-state compression makes the model vulnerable to information dilution, particularly when the useful signal is sparse, transient, and embedded in substantial microstructure noise.

2.4 Final Sequence Model: LSTM with Engineered Features

To address the limitations of the vanilla RNN, our final sequence architecture employs a Long Short-Term Memory (LSTM) network. Relative to a standard RNN, the LSTM is better suited to this task because its gating structure allows it to regulate what information is written into memory, what is preserved, and what is discarded. This is especially important in our setting, where predictive content may arise from short-lived but economically meaningful local dynamics, such as the buildup of order-flow pressure, spread dislocations, or temporary liquidity imbalances.

A key design choice is that we do not feed raw tick-by-tick price and size streams directly into the LSTM. Instead, the sequence model operates on the engineered features constructed in the previous section, including pricing variables, market microstructure measures, and broader

Bitcoin regime indicators. This hybrid design combines domain-driven financial feature engineering with temporal representation learning. The engineered variables provide a structured and economically interpretable state description, while the LSTM learns how these variables evolve jointly over the recent 60-second window.

This architecture is well aligned with the structure of the prediction problem. The model is not asked to rediscover basic financial transformations from noisy raw observations; rather, it is asked to learn how already meaningful economic signals interact over time. In that sense, the LSTM does not replace feature engineering, but extends it by modeling the nonlinear temporal dependence among economically structured inputs. Empirically, this hybrid architecture provides the strongest overall performance among the model classes considered, suggesting that short-horizon predictability in this market is driven not only by cross-sectional feature levels, but also by their recent path of evolution.

3. Model statistical data (Out of Sample)

Baseline: XGBoost

RMSE: 0.327701
MAE: 0.070471
R²: 0.000590
Rank IC: 0.1223
Directional accuracy: 50.77%

1.lstm

RMSE: 0.341964
MAE: 0.072208
R²: -0.0028
Rank IC: 0.2209
Directional Accuracy: 60.19%

Win rate:52.16%
Net pnl: 283.2191

2.rnn

RMSE: 0.341496
MAE: 0.072100
R²: -0.000028
Rank IC: 0.1954
Directional accuracy: 59.26%

Win rate:49.02%
Net pnl: -13.7652

3. mlp

RMSE: 0.327518
MAE: 0.069295
R²: 0.001710
Pearson IC: 0.0491
Rank IC: 0.2025

Directional accuracy: 59.48%

Win rate:50.94%

Net pnl: 185.8978